

The topology of software risk in scientific models

5. Dynamic profiling validation

Arnald Puy

Contents

1	Preliminary	2
2	Helpers and model registry	3
3	Pre-registration	8
4	Static-dynamic join	9
5	Analysis 1: Path coverage	11
5.1	Unstratified contrast	11
5.2	Within-length deciles (Analysis 1b)	12
6	Analysis 2: Rank correlation between P_k and E_k	17
7	Analysis 3: Reverse predictive check	19
8	Analysis 4: Matched-frequency contrast	21
9	Analysis 5: Dormant high-risk paths	24
10	Cross-model synthesis	27
11	Session information	32

1 Preliminary

This document layers a dynamic-profiling validation on top of the static path-level software-risk framework built in `code_hydrological_models.Rmd`. For each model with available runtime traces (currently ORCHIDEE and PCR-GLOBWB; VIC and HYPE expected), it joins the static call graph to the per-edge runtime call counts and runs the five analyses pre-registered in `dynamic_validation/pre_registration.md`:

1. **Path coverage.** Are statically high-risk paths preferentially exercised at runtime?
2. **Rank correlation.** Among exercised paths, does P_k order runtime intensity E_k^{sum} correctly?
3. **Reverse predictive check.** Restricting to runtime-heavy paths, does static P_k still discriminate on the underlying node-level metrics?
4. **Matched-frequency contrast.** Does static topology add information beyond execution frequency, when high- and low- P_k paths are matched on E_k^{sum} , length and statement count?
5. **Dormant high-risk paths.** Which statically high-risk paths are not (fully) exercised in the chosen configuration?

The individual analyses also live as standalone scripts under `dynamic_validation/analysis/` for ad-hoc re-runs and as a development view. This `.Rmd` is the canonical reference: every number reported in `dynamic_validation/summary.md` and `dynamic_validation/draft_results_paragraph.md` is reproduced here.

```
# PRELIMINARY FUNCTIONS #####
#####

sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",
                          "cowplot", "readxl", "ggrepel", "tidytext", "here",
                          "tidygraph", "igraph", "foreach", "parallel", "ggraph",
                          "tools", "purrr", "sensobol", "benchmarkme", "softwareRisk",
                          "boot", "MatchIt", "effsize"))

# Set seed -----

seed <- 123
set.seed(seed)
```

2 Helpers and model registry

This section bundles the data loaders, the model registry (which models have runtime data and which language each was profiled in), and the per-path helpers used throughout. Mirrors `dynamic_validation/analysis/00_setup.R`.

```
# MODEL REGISTRY #####
# Append rows here when VIC and HYPE arrive. `model` matches keys in
# full_paths_df.xlsx / full_node_df.xlsx; `raw_subdir` is the folder under
# dynamic_validation/.

model_registry <- data.table(
  model      = c("ORCHIDEE", "PCR-GLOBWB"),
  raw_subdir = c("ORCHIDEE", "PCR"),
  lang       = c("fortran", "python")
)

repo_root <- getwd()
dv_dir    <- file.path(repo_root, "dynamic_validation")
raw_dir   <- function(model) file.path(dv_dir, model)
out_dir   <- file.path(dv_dir, "results", "per_model")
syn_dir   <- file.path(dv_dir, "results", "synthesis")
dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)
dir.create(syn_dir, recursive = TRUE, showWarnings = FALSE)

available_models <- function() {
  reg <- copy(model_registry)
  reg[, available := vapply(raw_subdir, function(s) {
    file.exists(file.path(raw_dir(s), "runtime_callgraph_edges.csv"))
  }, logical(1))]
  reg[available == TRUE]
}

# NAME NORMALISATION #####
# Both static and runtime caller / callee names are lowercased and stripped of
# a leading class qualifier. Adjust here if model-specific quirks emerge
# (Fortran name mangling, Python decorators, etc.).

norm_fun_name <- function(x) {
  out <- tolower(trimws(as.character(x)))
  out <- sub(".*\\.\\.", "", out)
  out
}

# STATIC SIDE LOADERS #####
# Read the canonical outputs of code_hydrological_models.R.

load_full_paths_df <- function() {
```

```

f <- file.path(repo_root, "full_paths_df.xlsx")
if (!file.exists(f)) stop("Missing ", f,
  " - run code_hydrological_models.R first.")
as.data.table(read.xlsx(f))
}

load_full_nodes_df <- function() {
  f <- file.path(repo_root, "full_nodes_df.xlsx")
  if (!file.exists(f)) stop("Missing ", f,
    " - run code_hydrological_models.R first.")
  as.data.table(read.xlsx(f))
}

load_full_node_df <- function() {
  f <- file.path(repo_root, "full_node_df.xlsx")
  if (!file.exists(f)) stop("Missing ", f,
    " - run code_hydrological_models.R first.")
  as.data.table(read.xlsx(f))
}

# Per-path uncertainty summary - prefers the slim full_ua_summary.csv
# (produced by dynamic_validation/analysis/00b_extract_ua_summary.R; ~8.7 MB)
# and falls back to the raw 1.78 GB full_ua_df.csv with an sd approximated
# from quantiles.

load_uncertainty_summary <- function(models = NULL) {
  slim <- file.path(repo_root, "full_ua_summary.csv")
  raw <- file.path(repo_root, "full_ua_df.csv")
  if (file.exists(slim)) {
    ua <- fread(slim)
  } else if (file.exists(raw)) {
    cols <- c("model", "path_id", "path_nodes", "path_str", "hops",
      "P_k_min", "P_k_mean", "P_k_max",
      "P_k_q025", "P_k_q50", "P_k_q975")
    ua <- fread(raw, select = cols)
    ua[, P_k_sd := (P_k_q975 - P_k_q025) / (2 * 1.96)]
    ua[, CV_k := P_k_sd / pmax(P_k_mean, 1e-12)]
  } else {
    stop("Missing both ", slim, " and ", raw,
      " - run code_hydrological_models.R UA section, then ",
      "dynamic_validation/analysis/00b_extract_ua_summary.R.")
  }
  if (!is.null(models)) ua <- ua[model %in% models]
  if (!"risk_form" %in% names(ua)) ua[, risk_form := "additive"]
  ua[]
}

```

```

load_static_edges <- function(model_label, lang) {
  fn <- sprintf("call_metrics_%s_%s.csv", lang, model_label)
  f <- file.path(repo_root, "datasets", "call_metrics", fn)
  if (!file.exists(f)) stop("Missing ", f)
  dt <- fread(f)
  setnames(dt, c("function", "call"), c("caller", "callee"), skip_absent = TRUE)
  dt[, `:=`(
    caller_norm = norm_fun_name(caller),
    callee_norm = norm_fun_name(callee),
    model       = model_label,
    lang        = lang
  )]
  unique(dt[, .(caller_norm, callee_norm, model, lang)])
}

# DYNAMIC SIDE LOADER #####
# runtime_callgraph_edges.csv schema (as of 2026-05-17): model, lang, caller,
# callee, caller_file, caller_line, callee_file_hint, callee_kind,
# runtime_calls. Name normalisation can collapse multiple raw edges into the
# same (caller_norm, callee_norm); sum the runtime_calls to preserve intensity.

load_runtime_edges <- function(model_label, raw_subdir) {
  f <- file.path(raw_dir(raw_subdir), "runtime_callgraph_edges.csv")
  if (!file.exists(f)) stop("Missing ", f)
  dt <- fread(f)
  dt[, `:=`(
    caller_norm = norm_fun_name(caller),
    callee_norm = norm_fun_name(callee)
  )]
  if (!"runtime_calls" %in% names(dt)) dt[, runtime_calls := NA_integer_]
  dt[, .(runtime_calls = sum(runtime_calls, na.rm = TRUE)),
    by = .(caller_norm, callee_norm)]
}

# PER-PATH HELPERS #####
# path_nodes in full_paths_df is a delimited string after the xlsx round-trip.
# split_path_nodes returns the ordered node sequence; path_to_edges turns it
# into a per-edge table; aggregate_Ek collapses runtime counts along a path.

split_path_nodes <- function(path_nodes_str) {
  for (sep in c(" -> ", ", ", "|")) {
    if (any(grepl(sep, path_nodes_str, fixed = TRUE))) {
      return(strsplit(path_nodes_str, sep, fixed = TRUE))
    }
  }
  strsplit(path_nodes_str, "\\s+")
}

```

```

path_to_edges <- function(node_seq) {
  if (length(node_seq) < 2L) {
    return(data.table(caller_norm = character(), callee_norm = character()))
  }
  data.table(
    caller_norm = norm_fun_name(node_seq[-length(node_seq)]),
    callee_norm = norm_fun_name(node_seq[-1L])
  )
}

aggregate_Ek <- function(edge_dt) {
  rc <- edge_dt$runtime_calls
  rc[is.na(rc)] <- 0
  if (length(rc) == 0L) return(list(Ek_min = NA_real_, Ek_sum = NA_real_))
  list(Ek_min = min(rc), Ek_sum = sum(rc))
}

# Expand each static path into one row per (path_id, position, node) for a
# given (model, risk_form), with node-level attributes (cyclomatic_complexity,
# indeg, btw, risk_score, file) attached. Used by Analyses 3, 4, 5.

expand_paths_to_nodes <- function(model_label,
                                   risk_form_label = "additive",
                                   paths_df = NULL,
                                   node_df = NULL) {
  if (is.null(paths_df)) paths_df <- load_full_paths_df()
  if (is.null(node_df)) node_df <- load_full_node_df()

  fp <- paths_df[model == model_label & risk_form == risk_form_label]
  fn <- node_df[model == model_label,
                .(model, name, file, cyclomatic_complexity, indeg, btw,
                  risk_score)]
  if (!nrow(fp)) return(data.table())

  per_path <- rbindlist(lapply(seq_len(nrow(fp)), function(i) {
    nodes <- split_path_nodes(fp$path_nodes[i])[[1]]
    if (!length(nodes)) return(NULL)
    data.table(model = model_label, path_id = fp$path_id[i],
               pos = seq_along(nodes), name = nodes)
  }), use.names = TRUE, fill = TRUE)

  merge(per_path, fn, by = c("model", "name"), all.x = TRUE)
}

cat("[setup] models with runtime data:\n")

```

```
## [setup] models with runtime data:
```

```
print(available_models())
```

```
##          model raw_subdir    lang available
##          <char>    <char> <char>    <lgcl>
## 1:  ORCHIDEE  ORCHIDEE fortran      TRUE
## 2: PCR-GLOBWB      PCR  python      TRUE
```

3 Pre-registration

The decisions in `dynamic_validation/pre_registration.md` were locked before any analysis was run against the dynamic data. Highlights relevant to interpreting the numbers below:

- **Path-level aggregation of E_k .** Primary aggregator: `sum` (originally `min`; switched on 2026-05-17 because only 38 of 3,152 ORCHIDEE paths had every edge exercised at least once, making `min` collapse to a near-binary indicator informationally identical to `all_exercised`). `min` is reported alongside as a robustness check in Analysis 2.
- **High-risk stratum:** top decile of P_k AND $CV(P_k)$ below the model-specific median CV. Matches the bug-injection filter in the manuscript.
- **Low-risk stratum:** bottom decile of P_k (no CV filter).
- **Effect-size convention:** common-language A (Vargha & Delaney), matching Table S2 of the manuscript.
- **Predictive threshold:** $A > 0.55$ at $p < 0.05$ within a given comparison; the global decision rule for “framework predicts at runtime” additionally requires Spearman $\rho > 0$ with bootstrap CI strictly above zero, in at least three of the four target models.

```
# Decision parameters used throughout - all locked in pre_registration.md.

TOP_DECILE      <- 0.90
BOTTOM_DECILE   <- 0.10
RISK_FORM_PRIMARY <- "additive"
EK_PRIMARY      <- "Ek_sum"    # see deviation log 2026-05-17
EK ROBUSTNESS   <- "Ek_min"
N_BOOT          <- 1000L
A_THRESHOLD     <- 0.55
P_THRESHOLD     <- 0.05
TOP_K_FUNCTIONS <- 3L
MIN_PATHS_PER_BIN <- 20L # within-length deciles require at least this many

color_strata <- c("top_decile" = "#C65D09", "bottom_decile" = "#3B5B92")
```


4 Static-dynamic join

For each model with available runtime data, build a per-path joined table containing:

- the static path identifiers and risk summary (P_k_q50, CV_k, hops),
- per-path counts of edges exercised at runtime under the looser (“any”) and stricter (“all”) definitions,
- per-path runtime intensity under both aggregators (Ek_min, Ek_sum), with unmatched edges contributing zero calls.

```
# JOIN STATIC + DYNAMIC #####
#####

ua_summary  <- load_uncertainty_summary(models = available_models())$model
paths_df    <- load_full_paths_df()
node_df     <- load_full_node_df()
nodes_df    <- load_full_nodes_df()

join_one_model <- function(model_label, raw_subdir, lang) {

  static_edges <- load_static_edges(model_label, lang)
  runtime_edges <- load_runtime_edges(model_label, raw_subdir)

  matched_edges <- merge(static_edges, runtime_edges,
                        by = c("caller_norm", "callee_norm"),
                        all.x = TRUE)
  matched_edges[, exercised := !is.na(runtime_calls)]
  setkey(matched_edges, caller_norm, callee_norm)

  paths <- ua_summary[model == model_label]

  per_path <- paths[, {
    node_seq <- split_path_nodes(path_nodes)[[1]]
    edges     <- path_to_edges(node_seq)
    if (nrow(edges) == 0L) {
      list(n_edges = 0L, n_edges_static = 0L, n_edges_exercised = 0L,
           frac_exercised = NA_real_, any_exercised = NA, all_exercised = NA,
           Ek_min = NA_real_, Ek_sum = NA_real_)
    } else {
      joined <- matched_edges[edges, on = c("caller_norm", "callee_norm")]
      ex     <- !is.na(joined$exercised) & joined$exercised
      Ek     <- aggregate_Ek(joined)
      list(n_edges = nrow(edges),
           n_edges_static = sum(!is.na(joined$model)),
           n_edges_exercised = sum(ex),
           frac_exercised = sum(ex) / nrow(edges),
           any_exercised = any(ex),
           all_exercised = all(ex),
           Ek_min = Ek$Ek_min,
```

```

        Ek_sum = Ek$Ek_sum)
    }
  }, by = .(model, path_id, risk_form, hops,
            P_k_min, P_k_mean, P_k_max, P_k_q025, P_k_q50, P_k_q975, CV_k)]

  out_f <- file.path(out_dir, paste0(model_label, "_paths_joined.csv"))
  fwrite(per_path, out_f)

  diag <- data.table(
    model      = model_label,
    lang       = lang,
    static_edges = nrow(static_edges),
    runtime_edges = nrow(runtime_edges),
    matched_edges = sum(matched_edges$exercised),
    runtime_coverage_pct =
      100 * sum(matched_edges$exercised) / nrow(static_edges)
  )
  fwrite(diag, file.path(out_dir, paste0(model_label, "_edge_diagnostics.csv")))
  list(paths = per_path, diag = diag)
}

reg <- available_models()
join_results <- lapply(seq_len(nrow(reg)), function(i) {
  join_one_model(reg$model[i], reg$raw_subdir[i], reg$lang[i])
})
names(join_results) <- reg$model

# Pool joined tables for downstream analysis
all_joined <- rbindlist(lapply(join_results, `[`, "paths"))

# Edge-level coverage diagnostics
edge_diag <- rbindlist(lapply(join_results, `[`, "diag"))
print(edge_diag)

```

```

##      model      lang static_edges runtime_edges matched_edges
##      <char>  <char>      <int>         <int>         <int>
## 1:  ORCHIDEE  fortran      1368           767           168
## 2:  PCR-GLOBWB python       116           144            9
##      runtime_coverage_pct
##      <num>
## 1:      12.280702
## 2:      7.758621

```

The edge-level diagnostics quantify how much of each model's static call graph the runtime trace actually reached. They are a precondition for every downstream coverage claim: if the matched-edge fraction is small, the dormancy numbers below are bounded by it.

5 Analysis 1: Path coverage

5.1 Unstratified contrast

Top decile vs bottom decile of P_k , three coverage metrics:

- `frac_any`: at least one edge of the path was exercised (loose).
- `frac_edge`: mean fraction of edges per path that were exercised (length-insensitive).
- `frac_all`: every edge of the path was exercised (strict).

```
# ANALYSIS 1 - PATH COVERAGE (UNSTRATIFIED) #####
#####

coverage_dt <- all_joined[
  risk_form == RISK_FORM_PRIMARY & !is.na(P_k_q50),
  {
    qhi    <- quantile(P_k_q50, TOP_DECILE,    na.rm = TRUE)
    qlo    <- quantile(P_k_q50, BOTTOM_DECILE, na.rm = TRUE)
    cv_med <- median(CV_k, na.rm = TRUE)
    hi_idx <- P_k_q50 >= qhi & CV_k < cv_med
    lo_idx <- P_k_q50 <= qlo
    rbind(
      data.table(stratum = "top_decile",
                 n_paths = sum(hi_idx),
                 frac_any = mean(any_exercised[hi_idx], na.rm = TRUE),
                 frac_all = mean(all_exercised[hi_idx], na.rm = TRUE),
                 frac_edge = mean(frac_exercised[hi_idx], na.rm = TRUE),
                 median_hops = as.numeric(median(hops[hi_idx], na.rm = TRUE))),
      data.table(stratum = "bottom_decile",
                 n_paths = sum(lo_idx),
                 frac_any = mean(any_exercised[lo_idx], na.rm = TRUE),
                 frac_all = mean(all_exercised[lo_idx], na.rm = TRUE),
                 frac_edge = mean(frac_exercised[lo_idx], na.rm = TRUE),
                 median_hops = as.numeric(median(hops[lo_idx], na.rm = TRUE)))
    )
  },
  by = .(model, risk_form)
]

fwrite(coverage_dt, file.path(syn_dir, "01_coverage_table.csv"))
print(coverage_dt)
```

##	model	risk_form	stratum	n_paths	frac_any	frac_all	frac_edge
##	<char>	<char>	<char>	<int>	<num>	<num>	<num>
## 1:	ORCHIDEE	additive	top_decile	318	1.0000000	0.0000000	0.45888989
## 2:	ORCHIDEE	additive	bottom_decile	316	0.1234177	0.05696203	0.08728903
## 3:	PCR-GLOBWB	additive	top_decile	13	0.9230769	0.07692308	0.45512821
## 4:	PCR-GLOBWB	additive	bottom_decile	15	0.0000000	0.0000000	0.00000000
##	median_hops						

```
##          <num>
## 1:      7
## 2:      1
## 3:      4
## 4:      1

coverage_long <- melt(
  coverage_dt,
  id.vars      = c("model", "risk_form", "stratum", "n_paths", "median_hops"),
  measure.vars = c("frac_any", "frac_edge", "frac_all"),
  variable.name = "metric",
  value.name    = "value"
)
coverage_long[, metric := factor(metric,
  levels = c("frac_any", "frac_edge", "frac_all"),
  labels = c("any edge", "mean fraction of edges", "all edges"))]

ggplot(coverage_long,
  aes(x = model, y = value, fill = stratum)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  facet_wrap(~ metric) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1),
    limits = c(0, 1)) +
  scale_fill_manual(values = color_strata,
    labels = c("bottom decile P_k", "top decile P_k")) +
  labs(x = NULL, y = "coverage", fill = NULL) +
  theme_minimal(base_size = 11) +
  theme(legend.position = "bottom")
```

The unstratified contrast is large but partly driven by the strong correlation between P_k and path length under the saturating-OR aggregator (Eq. 5 of the manuscript). The length-controlled version follows.

5.2 Within-length deciles (Analysis 1b)

For each path-length bin separately, take the top vs bottom decile of P_k within the bin and compare runtime coverage. Length is held constant by construction; the within-bin Wilcoxon test plus the common-language effect size A summarise discrimination.

```
# ANALYSIS 1b - WITHIN-LENGTH DECILES + WILCOXON #####
#####

within_length_dt <- all_joined[risk_form == RISK_FORM_PRIMARY & !is.na(P_k_q50)]
within_length_dt[, hops_bin := cut(hops,
  breaks = c(0, 1, 2, 4, 7, Inf),
  labels = c("1", "2", "3-4", "5-7", "8+"),
  right = TRUE)]

within_decile_dt <- within_length_dt[
```

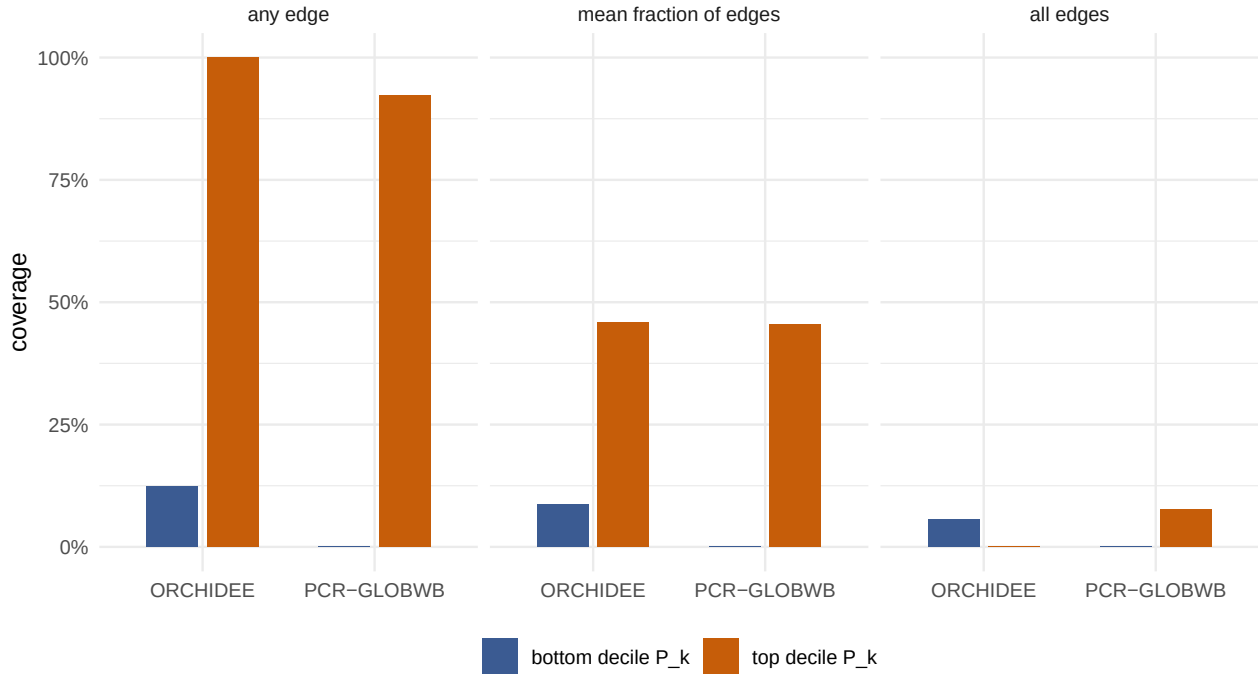


Figure 1: Analysis 1 unstratified: fraction of paths exercised at runtime by static-risk stratum, under three coverage metrics. Note the median-hops column in the table - top-decile paths are substantially longer than bottom-decile paths, which inflates the **any** contrast.

```

,
{
  if (.N < MIN_PATHS_PER_BIN || length(unique(P_k_q50)) < 2L) {
    data.table()
  } else {
    qhi <- quantile(P_k_q50, TOP_DECILE, na.rm = TRUE)
    qlo <- quantile(P_k_q50, BOTTOM_DECILE, na.rm = TRUE)
    top <- P_k_q50 >= qhi
    bot <- P_k_q50 <= qlo
    rbind(
      data.table(stratum = "top_decile",
        n_paths = sum(top),
        frac_any = mean(any_exercised[top], na.rm = TRUE),
        frac_edge = mean(frac_exercised[top], na.rm = TRUE),
        frac_all = mean(all_exercised[top], na.rm = TRUE)),
      data.table(stratum = "bottom_decile",
        n_paths = sum(bot),
        frac_any = mean(any_exercised[bot], na.rm = TRUE),
        frac_edge = mean(frac_exercised[bot], na.rm = TRUE),
        frac_all = mean(all_exercised[bot], na.rm = TRUE))
    )
  }
},
by = .(model, risk_form, hops_bin)

```

```

]

within_test_dt <- within_length_dt[
  ,
  {
    if (.N < MIN_PATHS_PER_BIN || length(unique(P_k_q50)) < 2L) {
      data.table()
    } else {
      qhi <- quantile(P_k_q50, TOP_DECILE, na.rm = TRUE)
      qlo <- quantile(P_k_q50, BOTTOM_DECILE, na.rm = TRUE)
      x_top <- frac_exercised[P_k_q50 >= qhi]
      x_bot <- frac_exercised[P_k_q50 <= qlo]
      if (length(x_top) < 3L || length(x_bot) < 3L) {
        data.table()
      } else {
        w <- suppressWarnings(wilcox.test(x_top, x_bot))
        A <- tryCatch(effsize::VD.A(x_top, x_bot)$estimate,
                      error = function(e) NA_real_)
        data.table(n_top = length(x_top), n_bot = length(x_bot),
                   median_top = median(x_top, na.rm = TRUE),
                   median_bot = median(x_bot, na.rm = TRUE),
                   wilcox_p = w$p.value, effect_A = A)
      }
    }
  },
  by = .(model, risk_form, hops_bin)
]

within_test_dt[, signif := fcase(
  is.na(wilcox_p), "",
  wilcox_p < 0.001, "****",
  wilcox_p < 0.01,  "***",
  wilcox_p < 0.05,  "**",
  default = "ns"
)]

fwrite(within_decile_dt, file.path(syn_dir, "01b_within_length_deciles.csv"))
fwrite(within_test_dt,   file.path(syn_dir, "01b_within_length_tests.csv"))

print(within_decile_dt)

```

##	model	risk_form	hops_bin	stratum	n_paths	frac_any	frac_edge
##	<char>	<char>	<fctr>	<char>	<int>	<num>	<num>
## 1:	ORCHIDEE	additive	1	top_decile	23	0.04347826	0.04347826
## 2:	ORCHIDEE	additive	1	bottom_decile	24	0.58333333	0.58333333
## 3:	ORCHIDEE	additive	2	top_decile	45	0.06666667	0.03333333
## 4:	ORCHIDEE	additive	2	bottom_decile	45	0.06666667	0.03333333
## 5:	ORCHIDEE	additive	3-4	top_decile	100	0.95000000	0.44500000

```
## 6: ORCHIDEE additive 3-4 bottom_decile 100 0.39000000 0.19000000
## 7: ORCHIDEE additive 5-7 top_decile 133 1.00000000 0.47708557
## 8: ORCHIDEE additive 5-7 bottom_decile 133 0.67669173 0.17160759
## 9: ORCHIDEE additive 8+ top_decile 16 1.00000000 0.45902778
## 10: ORCHIDEE additive 8+ bottom_decile 16 0.81250000 0.23437500
## 11: PCR-GLOBWB additive 1 top_decile 7 0.00000000 0.00000000
## 12: PCR-GLOBWB additive 1 bottom_decile 15 0.00000000 0.00000000
##      frac_all
##      <num>
## 1: 0.043478261
## 2: 0.583333333
## 3: 0.000000000
## 4: 0.000000000
## 5: 0.000000000
## 6: 0.010000000
## 7: 0.000000000
## 8: 0.007518797
## 9: 0.000000000
## 10: 0.000000000
## 11: 0.000000000
## 12: 0.000000000
```

```
print(within_test_dt)
```

```
##      model risk_form hops_bin n_top n_bot median_top median_bot      wilcox_p
##      <char>      <char>      <fctr> <int> <int>      <num>      <num>      <num>
## 1: ORCHIDEE additive      1      23      24 0.0000000      1.00 9.103629e-05
## 2: ORCHIDEE additive      2      45      45 0.0000000      0.00 1.000000e+00
## 3: ORCHIDEE additive 3-4      100      100 0.5000000      0.00 2.849521e-12
## 4: ORCHIDEE additive 5-7      133      133 0.4285714      0.20 3.143054e-36
## 5: ORCHIDEE additive 8+      16      16 0.4444444      0.25 7.334440e-07
## 6: PCR-GLOBWB additive      1      7      15 0.0000000      0.00      NaN
##      effect_A signif
##      <num> <char>
## 1: 0.2300725 ***
## 2: 0.5000000 ns
## 3: 0.7762000 ***
## 4: 0.9311154 ***
## 5: 1.0000000 ***
## 6: 0.5000000
```

```
if (nrow(within_decile_dt)) {
  ggplot(within_decile_dt,
    aes(x = hops_bin, y = frac_edge, fill = stratum)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  facet_grid(risk_form ~ model) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1),
    limits = c(0, 1)) +
  scale_fill_manual(values = color_strata,
```

```

      labels = c("bottom decile", "top decile")) +
    labs(x = "path length (hops)", y = "mean fraction of edges exercised",
         fill = NULL) +
    theme_minimal(base_size = 11) +
    theme(legend.position = "bottom")
}

```

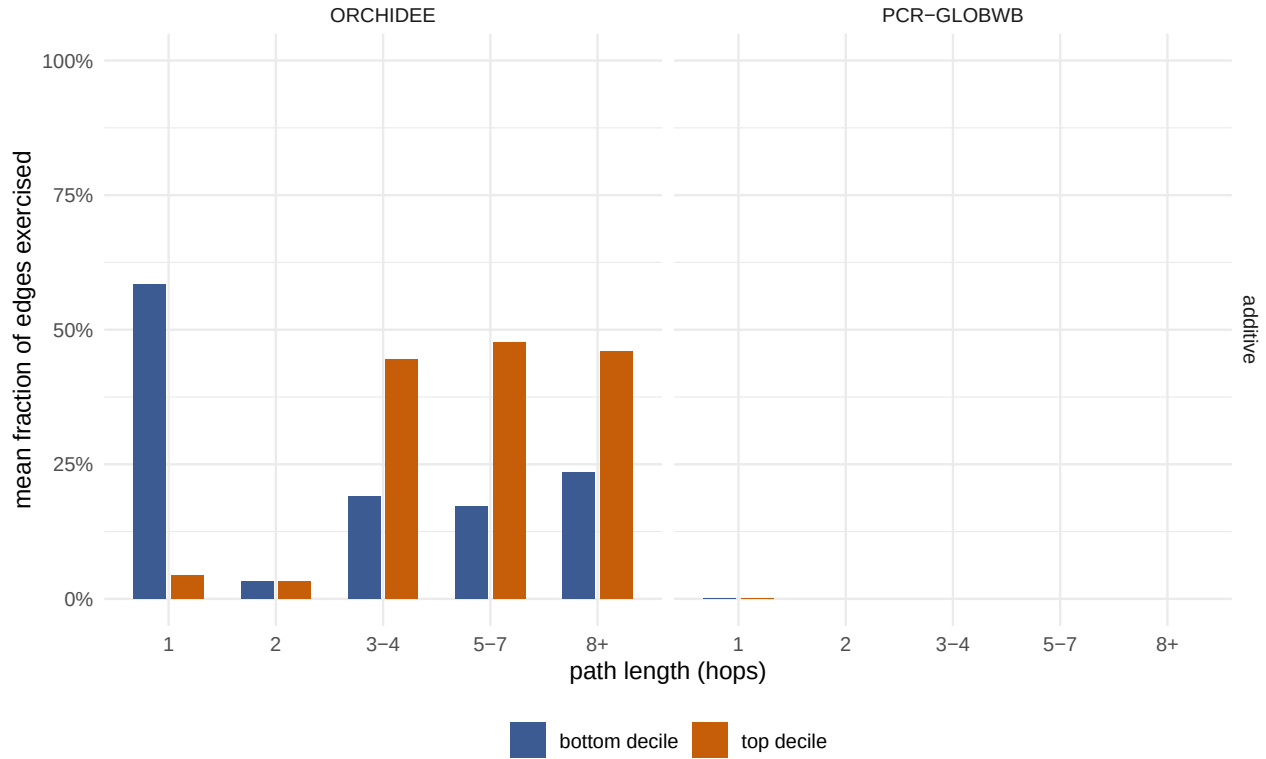


Figure 2: Analysis 1b: mean fraction of edges exercised per path, by hops bin and static-risk stratum. The decile cuts are taken WITHIN each hops bin so the top-vs-bottom comparison is length-controlled. For paths of 3 or more hops the framework discriminates strongly; for hops 1 the relationship reverses, consistent with the framework's design as a path-level integrator over multi-node chains.

6 Analysis 2: Rank correlation between P_k and E_k

Among paths that runtime actually exercised, does the static risk score P_k rank-order their runtime intensity? Spearman ρ on E_k^{sum} (primary) and on E_k^{min} (robustness; see deviation log). Bootstrap percentile 95 % CI with $N = 1000$ resamples.

```
# ANALYSIS 2 - RANK CORRELATION P_k vs E_k #####
#####

corr_results <- rbindlist(lapply(reg$model, function(m) {
  dt <- all_joined[model == m & risk_form == RISK_FORM_PRIMARY &
    any_exercised == TRUE &
    !is.na(P_k_q50) & !is.na(get(EK_PRIMARY))]
  if (!nrow(dt) || all(is.na(dt[[EK_PRIMARY]]))) return(NULL)

  rho <- suppressWarnings(cor(dt$P_k_q50, dt[[EK_PRIMARY]], method = "spearman"))
  tau <- suppressWarnings(cor(dt$P_k_q50, dt[[EK_PRIMARY]], method = "kendall"))
  rho_robust <- suppressWarnings(
    cor(dt$P_k_q50, dt[[EK_ROBUSTNESS]], method = "spearman"))

  boot_rho <- boot::boot(
    data      = dt,
    statistic = function(d, i) suppressWarnings(
      cor(d$P_k_q50[i], d[[EK_PRIMARY]][i], method = "spearman")),
    R = N_BOOT
  )
  ci <- tryCatch(
    boot::boot.ci(boot_rho, type = "perc")$percent[1, c(4, 5)],
    error = function(e) c(NA_real_, NA_real_)
  )

  data.table(model = m, n_paths = nrow(dt),
    spearman_sum = rho, kendall_sum = tau,
    rho_sum_ci_lo = ci[1], rho_sum_ci_hi = ci[2],
    spearman_min = rho_robust)
}), fill = TRUE)

fwrite(corr_results, file.path(syn_dir, "02_rank_correlation.csv"))
print(corr_results)
```

```
##      model n_paths spearman_sum kendall_sum rho_sum_ci_lo rho_sum_ci_hi
##      <char>  <int>      <num>      <num>      <num>      <num>
## 1:  ORCHIDEE   2100    0.7905821  0.6235002    0.7694998    0.8085537
## 2:  PCR-GLOBWB    17    0.1069692  0.1491125   -0.5136641    0.6152736
##      spearman_min
##      <num>
## 1:   -0.2093680
## 2:   -0.6231479
```

```

scatter_panels <- lapply(reg$model, function(m) {
  dt <- all_joined[model == m & risk_form == RISK_FORM_PRIMARY &
    any_exercised == TRUE & !is.na(get(EK_PRIMARY))]
  if (!nrow(dt)) return(NULL)
  rho_row <- corr_results[model == m]
  ggplot(dt, aes(x = P_k_q50, y = log10(get(EK_PRIMARY) + 1))) +
    geom_point(alpha = 0.4, size = 0.8) +
    geom_smooth(method = "loess", se = FALSE, colour = "#C65D09") +
    labs(title = sprintf("%s (rho_sum = %.2f [%.2f, %.2f], n = %d)",
      m, rho_row$spearman_sum,
      rho_row$rho_sum_ci_lo, rho_row$rho_sum_ci_hi,
      rho_row$n_paths),
      x = expression(P[k]~"(median across ensemble)"),
      y = expression(log[10]*("E[k]^{sum}+1"))) +
    theme_minimal(base_size = 10)
})
scatter_panels <- scatter_panels[!vapply(scatter_panels, is.null, logical(1))]
if (length(scatter_panels)) {
  cowplot::plot_grid(plotlist = scatter_panels, nrow = 1)
}

```

```

## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'

```

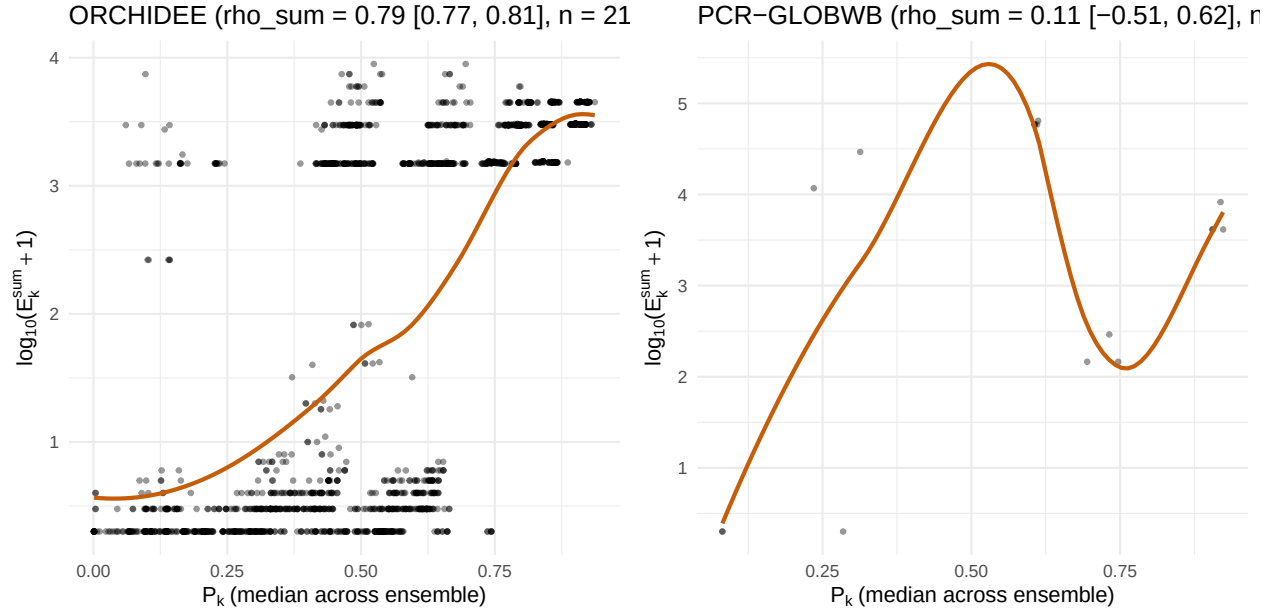


Figure 3: Analysis 2: P_k against $\log_{10}(E_k^{\text{sum}} + 1)$ for exercised paths in each model. Spearman rho is annotated per panel; loess in orange.

7 Analysis 3: Reverse predictive check

Restrict to the top decile of paths by E_k^{sum} (the runtime-heavy paths), then within that subset compare path-mean cyclomatic complexity, in-degree and betweenness between the top- P_k and bottom- P_k halves. Two-sample Wilcoxon plus common-language A . *Holding execution frequency fixed, does static topology still discriminate?*

```
# ANALYSIS 3 - REVERSE PREDICTIVE CHECK #####  
#####
```

```
TOP_RUNTIME_Q <- 0.90  
TOP_P_Q       <- 0.50 # split high vs low P_k at the median, within the  
                  # runtime-heavy subset  
METRICS       <- c("cyclomatic_complexity", "indeg", "btw")  
  
reverse_results <- list()  
reverse_plots  <- list()  
  
for (m in reg$model) {  
  
  dt <- all_joined[model == m & risk_form == RISK_FORM_PRIMARY &  
                  any_exercised == TRUE & !is.na(get(EK_PRIMARY))]  
  if (!nrow(dt) || all(is.na(dt[[EK_PRIMARY]]))) next  
  
  q_runtime <- quantile(dt[[EK_PRIMARY]], TOP_RUNTIME_Q, na.rm = TRUE)  
  hot_paths <- dt[get(EK_PRIMARY) >= q_runtime, .(model, path_id, P_k_q50)]  
  if (nrow(hot_paths) < 8L) next  
  
  per_path_nodes <- expand_paths_to_nodes(m, RISK_FORM_PRIMARY,  
                                         paths_df = paths_df,  
                                         node_df  = node_df)  
  
  per_path_metrics <- per_path_nodes[  
    path_id %in% hot_paths$path_id,  
    lapply(.SD, mean, na.rm = TRUE),  
    by = .(model, path_id),  
    .SDcols = METRICS  
  ]  
  merged <- merge(hot_paths, per_path_metrics, by = c("model", "path_id"))  
  if (nrow(merged) < 8L) next  
  
  q_pk <- quantile(merged$P_k_q50, TOP_P_Q)  
  merged[, stratum := ifelse(P_k_q50 >= q_pk, "high_Pk", "low_Pk")]  
  
  per_metric <- lapply(METRICS, function(mt) {  
    a <- merged[stratum == "high_Pk", get(mt)]  
    b <- merged[stratum == "low_Pk",  get(mt)]  
    if (!length(a) || !length(b)) return(NULL)  
    w <- suppressWarnings(wilcox.test(a, b))  
  })  
}
```

```

A <- tryCatch(effsize::VD.A(a, b)$estimate, error = function(e) NA_real_)
data.table(model = m, metric = mt,
            n_high = length(a), n_low = length(b),
            median_high = median(a, na.rm = TRUE),
            median_low = median(b, na.rm = TRUE),
            wilcox_p = w$p.value, effect_A = A)
})
reverse_results[[m]] <- rbindlist(per_metric, fill = TRUE)

plot_dt <- melt(merged, id.vars = c("model", "path_id", "stratum"),
               measure.vars = METRICS,
               variable.name = "metric", value.name = "path_mean")
reverse_plots[[m]] <- ggplot(plot_dt,
                             aes(x = stratum, y = path_mean, fill = stratum)) +
  geom_boxplot(alpha = 0.7, outlier.size = 0.5) +
  facet_wrap(~ metric, scales = "free_y", nrow = 1) +
  scale_fill_manual(values = c("high_Pk" = "#C65D09", "low_Pk" = "#3B5B92")) +
  labs(title = m, x = NULL, y = "path-mean") +
  theme_minimal(base_size = 10) + theme(legend.position = "none")
}

if (length(reverse_results)) {
  reverse_tbl <- rbindlist(reverse_results, fill = TRUE)
  fwrite(reverse_tbl, file.path(syn_dir, "03_reverse_predictive.csv"))
  print(reverse_tbl)
}

##      model                metric n_high n_low median_high median_low
##      <char>                <char> <int> <int>      <num>      <num>
## 1: ORCHIDEE cyclomatic_complexity   128   128         97         97
## 2: ORCHIDEE indeg                 128   128          0          0
## 3: ORCHIDEE btw                  128   128          0          0
##      wilcox_p effect_A
##      <num>      <num>
## 1: 0.003651277 0.586853
## 2:           NaN 0.500000
## 3:           NaN 0.500000

if (length(reverse_plots)) {
  cowplot::plot_grid(plotlist = reverse_plots, ncol = 1, align = "v")
}

```

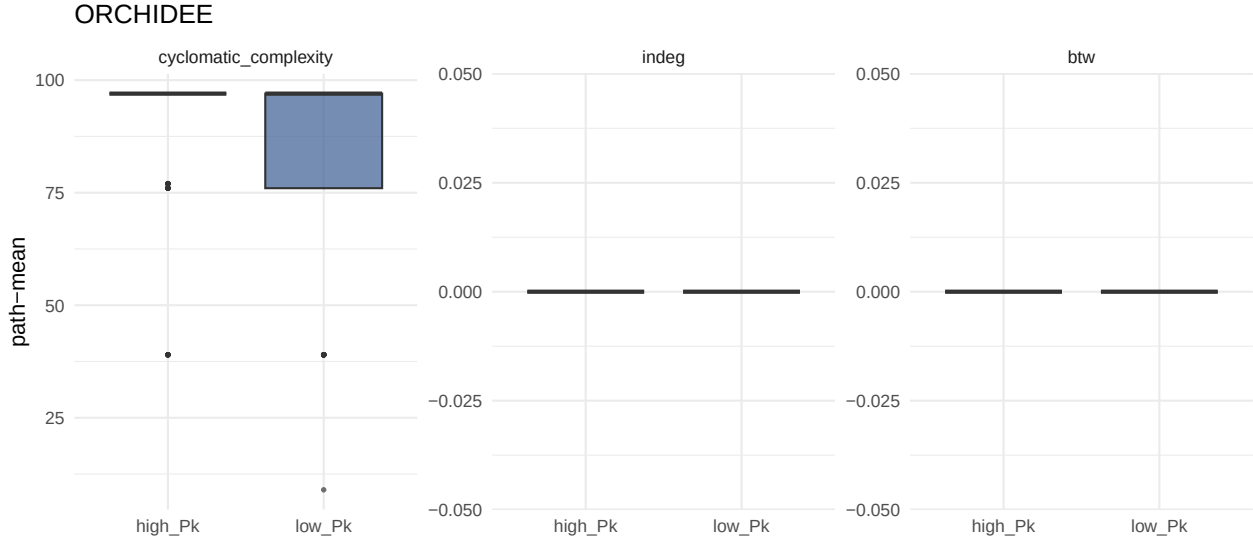


Figure 4: Analysis 3: distribution of path-mean static metrics (cyclomatic complexity, in-degree, betweenness) between high-Pk and low-Pk strata within the runtime-heavy decile, per model.

8 Analysis 4: Matched-frequency contrast

The cleanest test of “does static topology add information beyond execution frequency”: match each top-decile high- P_k path to a bottom-decile low- P_k path with comparable E_k^{sum} , hops and total statement count (Mahalanobis nearest-neighbour with replacement, `MatchIt`). Then compare static metrics on the matched set. Sample sizes below 15 pairs are flagged as exploratory.

```
# ANALYSIS 4 - MATCHED-FREQUENCY CONTRAST #####
#####
```

```
EXPLORATORY_THRESHOLD <- 15L
matched_results <- list()
diag_smd <- list()

for (m in reg$model) {

  dt <- all_joined[model == m & risk_form == RISK_FORM_PRIMARY &
    any_exercised == TRUE & !is.na(get(EK_PRIMARY))]
  if (!nrow(dt) || all(is.na(dt[[EK_PRIMARY]]))) next

  qhi <- quantile(dt$P_k_q50, TOP_DECILE, na.rm = TRUE)
  qlo <- quantile(dt$P_k_q50, BOTTOM_DECILE, na.rm = TRUE)
  cv_med <- median(dt$CV_k, na.rm = TRUE)

  dt[, group := fifelse(P_k_q50 >= qhi & CV_k < cv_med, 1L,
    fifelse(P_k_q50 <= qlo, 0L, NA_integer_))]
  dt_match <- dt[!is.na(group)]

  if (sum(dt_match$group == 1L) < 5L || sum(dt_match$group == 0L) < 5L) next
```

```

per_path_nodes <- expand_paths_to_nodes(m, RISK_FORM_PRIMARY,
                                       paths_df = paths_df,
                                       node_df  = node_df)

metric_dt <- per_path_nodes[
  path_id %in% dt_match$path_id,
  lapply(.SD, mean, na.rm = TRUE),
  by = .(path_id),
  .SDcols = METRICS
]
dt_match <- merge(dt_match, metric_dt, by = "path_id", all.x = TRUE)
dt_match[, statement_proxy := cyclomatic_complexity * hops]

mout <- tryCatch(
  MatchIt::matchit(
    group ~ get(EK_PRIMARY) + hops + statement_proxy,
    data   = dt_match,
    method = "nearest",
    distance = "mahalanobis",
    replace = TRUE
  ),
  error = function(e) { warning(e$message); NULL }
)
if (is.null(mout)) next

matched <- MatchIt::match.data(mout)
n_pairs <- sum(matched$group == 1L)

smd_pre  <- summary(mout)$sum.all[, "Std. Mean Diff."]
smd_post <- summary(mout)$sum.matched[, "Std. Mean Diff."]
diag_smd[[m]] <- data.table(
  model      = m,
  covariate  = names(smd_post),
  smd_pre    = smd_pre[names(smd_post)],
  smd_post   = smd_post
)

per_metric <- lapply(METRICS, function(mt) {
  a <- matched[[mt]][matched$group == 1L]
  b <- matched[[mt]][matched$group == 0L]
  if (!length(a) || !length(b)) return(NULL)
  w <- suppressWarnings(wilcox.test(a, b))
  A <- tryCatch(effsize::VD.A(a, b)$estimate, error = function(e) NA_real_)
  data.table(model = m, metric = mt, n_pairs = n_pairs,
             median_high = median(a, na.rm = TRUE),
             median_low  = median(b, na.rm = TRUE),
             wilcox_p     = w$p.value, effect_A = A,
             exploratory  = n_pairs < EXPLORATORY_THRESHOLD)
})

```

```

})
matched_results[[m]] <- rbindlist(per_metric, fill = TRUE)
}

if (length(matched_results)) {
  matched_tbl <- rbindlist(matched_results, fill = TRUE)
  fwrite(matched_tbl, file.path(syn_dir, "04_matched_contrast.csv"))
  fwrite(rbindlist(diag_smd, fill = TRUE),
         file.path(syn_dir, "04_matched_smd_diagnostics.csv"))
  print(matched_tbl)
}

```

```

##      model                metric n_pairs median_high median_low  wilcox_p
##      <char>                <char>  <int>      <num>      <num>      <num>
## 1: ORCHIDEE cyclomatic_complexity    211         77         12 0.05568453
## 2: ORCHIDEE          indeg          211          0          0         NaN
## 3: ORCHIDEE          btw           211          0          0         NaN
##      effect_A exploratory
##      <num>      <lgcl>
## 1: 0.8120063      FALSE
## 2: 0.5000000      FALSE
## 3: 0.5000000      FALSE

```

9 Analysis 5: Dormant high-risk paths

Of the within-length top-decile high-risk paths in each length bin, how many were not (fully) exercised in the chosen runtime configuration? *Loose* dormancy means no edge of the path fired; *strict* dormancy means at least one edge of the path did not fire. The within-length top-decile cut (rather than the overall top decile) ensures the dormant set is not just long paths that mechanically have more chances to fail.

```
# ANALYSIS 5 - DORMANT HIGH-RISK PATHS #####
#####

dormant_summary  <- list()
dormant_paths_out <- list()

for (m in reg$model) {

  dt <- all_joined[model == m & risk_form == RISK_FORM_PRIMARY & !is.na(P_k_q50)]
  if (!nrow(dt)) next
  dt[, hops_bin := cut(hops,
                       breaks = c(0, 1, 2, 4, 7, Inf),
                       labels = c("1", "2", "3-4", "5-7", "8+"),
                       right = TRUE)]

  hi <- dt[
    ,
    {
      if (.N < MIN_PATHS_PER_BIN || length(unique(P_k_q50)) < 2L) {
        data.table()
      } else {
        qhi    <- quantile(P_k_q50, TOP_DECILE, na.rm = TRUE)
        cv_med <- median(CV_k, na.rm = TRUE)
        .SD[P_k_q50 >= qhi & CV_k < cv_med]
      }
    },
    by = .(model, hops_bin)
  ]
  if (!nrow(hi)) next

  dormant_loose  <- hi[any_exercised == FALSE | is.na(any_exercised)]
  dormant_strict <- hi[all_exercised == FALSE | is.na(all_exercised)]

  dormant_summary[[m]] <- hi[
    , .(n_top_decile_in_bin = .N), by = .(model, hops_bin)
  ][
    merge(dormant_loose[, .N, by = .(model, hops_bin)],
          dormant_strict[, .N, by = .(model, hops_bin)],
          by = c("model", "hops_bin"), all = TRUE,
          suffixes = c("_loose", "_strict")),
```



```

on = c("model", "hops_bin"), nomatch = NA
][
, .(model, hops_bin,
  n_top_decile = n_top_decile_in_bin,
  n_dormant_loose = fcoalesce(N_loose, 0L),
  n_dormant_strict = fcoalesce(N_strict, 0L))
][
, `:=`(frac_dormant_loose = n_dormant_loose / n_top_decile,
  frac_dormant_strict = n_dormant_strict / n_top_decile)
]

if (!nrow(dormant_loose)) next

per_path_nodes <- expand_paths_to_nodes(m, RISK_FORM_PRIMARY,
  paths_df = paths_df,
  node_df = node_df)
per_path_nodes <- per_path_nodes[path_id %in% dormant_loose$path_id]
topk <- per_path_nodes[order(-risk_score),
  head(.SD, TOP_K_FUNCTIONS),
  by = .(model, path_id)]
topk_collapsed <- topk[, .(
  top_functions = paste(name, collapse = " | "),
  top_files = paste(unique(file), collapse = " | ")
), by = .(model, path_id)]
out <- merge(dormant_loose[, .(model, path_id, hops_bin, hops, P_k_q50, CV_k)],
  topk_collapsed, by = c("model", "path_id"), all.x = TRUE)
dormant_paths_out[[m]] <- out[order(hops_bin, -P_k_q50)]
}

if (length(dormant_summary)) {
  dormant_tbl <- rbindlist(dormant_summary, fill = TRUE)
  fwrite(dormant_tbl, file.path(syn_dir, "05_dormant_summary.csv"))
  fwrite(rbindlist(dormant_paths_out, fill = TRUE),
    file.path(syn_dir, "05_dormant_paths_table.csv"))
  print(dormant_tbl)
}

```

```

##      model hops_bin n_top_decile n_dormant_loose n_dormant_strict
##      <char>   <fctr>      <int>          <int>          <int>
## 1:  ORCHIDEE      1         1             1             1
## 2:  ORCHIDEE      2        41            39            41
## 3:  ORCHIDEE     3-4       100            5            100
## 4:  ORCHIDEE     5-7       133            0            133
## 5:  ORCHIDEE      8+        8             0             8
## 6:  PCR-GLOBWB     1         7             7             7
##      frac_dormant_loose frac_dormant_strict
##              <num>          <num>

```

## 1:	1.0000000	1
## 2:	0.9512195	1
## 3:	0.0500000	1
## 4:	0.0000000	1
## 5:	0.0000000	1
## 6:	1.0000000	1

10 Cross-model synthesis

A single headline table per model collects the within-length effect sizes, the rank correlation, the reverse predictive A on cyclomatic complexity, the matched A, and the within-length dormancy. The pre-registered decision rule (“framework predicts at runtime”) is evaluated last.

```
# CROSS-MODEL SYNTHESIS #####
#####

headline <- data.table(model = reg$model)

# Coverage
cov_wide <- dcast(coverage_dt, model ~ stratum,
                  value.var = c("frac_any", "frac_all", "frac_edge"))
headline <- merge(headline, cov_wide, by = "model", all.x = TRUE)

# Rank correlation
if (exists("corr_results")) {
  headline <- merge(headline,
                    corr_results[, .(model,
                                     spearman_Pk_Eksum = spearman_sum,
                                     spearman_ci_lo     = rho_sum_ci_lo,
                                     spearman_ci_hi     = rho_sum_ci_hi)],
                    by = "model", all.x = TRUE)
}

# Reverse predictive (cyclomatic complexity only - the discriminating metric)
if (length(reverse_results)) {
  rev_cc <- rbindlist(reverse_results, fill = TRUE)[metric == "cyclomatic_complexity",
                                                    .(model, A_reverse_cc = effect_A)]
  headline <- merge(headline, rev_cc, by = "model", all.x = TRUE)
}

# Matched contrast (cyclomatic complexity only)
if (length(matched_results)) {
  m_cc <- rbindlist(matched_results, fill = TRUE)[metric == "cyclomatic_complexity",
                                                  .(model, A_matched_cc = effect_A,
                                                    n_matched_pairs = n_pairs)]
  headline <- merge(headline, m_cc, by = "model", all.x = TRUE)
}

# Dormancy (one row per model summarising the hops 3-4 bin)
if (length(dormant_summary)) {
  d_one <- rbindlist(dormant_summary, fill = TRUE)[hops_bin == "3-4",
                                                  .(model,
                                                    dormant_loose_3_4 = frac_dormant_loose)]
  headline <- merge(headline, d_one, by = "model", all.x = TRUE)
}
```

```
fwrite(headline, file.path(syn_dir, "06_cross_model_headline.csv"))
print(headline)
```

```
## Key: <model>
##      model frac_any_bottom_decile frac_any_top_decile frac_all_bottom_decile
##      <char>          <num>          <num>          <num>
## 1:  ORCHIDEE          0.1234177          1.0000000          0.05696203
## 2:  PCR-GLOBWB          0.0000000          0.9230769          0.00000000
##      frac_all_top_decile frac_edge_bottom_decile frac_edge_top_decile
##      <num>          <num>          <num>
## 1:          0.00000000          0.08728903          0.4588899
## 2:          0.07692308          0.00000000          0.4551282
##      spearman_Pk_Eksum spearman_ci_lo spearman_ci_hi A_reverse_cc A_matched_cc
##      <num>          <num>          <num>          <num>          <num>
## 1:          0.7905821          0.7694998          0.8085537          0.586853          0.8120063
## 2:          0.1069692         -0.5136641          0.6152736              NA              NA
##      n_matched_pairs dormant_loose_3_4
##      <int>          <num>
## 1:          211          0.05
## 2:          NA          NA
```

```
# Pre-registered decision rule per model
if (exists("corr_results") && length(reverse_results) && length(matched_results)) {
  cor_ok <- corr_results[, .(model,
                             spearman_pos = spearman_sum > 0 & rho_sum_ci_lo > 0)]
  rev_ok <- rbindlist(reverse_results, fill = TRUE)[,
    .(any_A_above_reverse = any(effect_A > A_THRESHOLD, na.rm = TRUE)),
    by = model]
  mat_ok <- rbindlist(matched_results, fill = TRUE)[,
    .(any_A_above_matched = any(effect_A > A_THRESHOLD, na.rm = TRUE)),
    by = model]
  decision <- merge(cor_ok, rev_ok, by = "model", all = TRUE)
  decision <- merge(decision, mat_ok, by = "model", all = TRUE)
  decision[, supports_framework :=
    spearman_pos & (any_A_above_reverse | any_A_above_matched)]
  fwrite(decision, file.path(syn_dir, "06_decision_rule.csv"))
  print(decision)
  n_support <- sum(decision$supports_framework, na.rm = TRUE)
  cat(sprintf("\n[decision rule] %d of %d available models meet the joint criterion. ",
    n_support, nrow(decision)))
  cat("Full pre-registered claim requires >= 3 of 4 target models.\n")
}
```

```
## Key: <model>
##      model spearman_pos any_A_above_reverse any_A_above_matched
##      <char>          <lgcl>          <lgcl>          <lgcl>
## 1:  ORCHIDEE          TRUE          TRUE          TRUE
## 2:  PCR-GLOBWB          FALSE          NA          NA
```

```

##      supports_framework
##      <lgcl>
## 1:      TRUE
## 2:      FALSE
##
## [decision rule] 1 of 2 available models meet the joint criterion. Full pre-registered claim

panels <- list()

p_cov <- ggplot(coverage_dt,
               aes(x = model, y = frac_edge, fill = stratum)) +
  geom_col(position = position_dodge(0.7), width = 0.6) +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_manual(values = color_strata,
                    labels = c("bottom decile", "top decile")) +
  labs(title = "Coverage gap (mean frac. of edges exercised)",
       x = NULL, y = NULL, fill = NULL) +
  theme_minimal(base_size = 10) + theme(legend.position = "bottom")
panels <- c(panels, list(p_cov))

if (exists("corr_results") && nrow(corr_results)) {
  p_corr <- ggplot(corr_results, aes(x = model, y = spearman_sum)) +
    geom_pointrange(aes(ymin = rho_sum_ci_lo, ymax = rho_sum_ci_hi)) +
    geom_hline(yintercept = 0, linetype = "dashed") +
    labs(title = expression("Spearman " ~ rho ~ "(P"[k] ~ ", E"[k]^{\sum} ~ ")"),
         x = NULL, y = NULL) +
    theme_minimal(base_size = 10)
  panels <- c(panels, list(p_corr))
}

if (length(reverse_results) || length(matched_results)) {
  effect_dt <- rbind(
    if (length(reverse_results))
      rbindlist(reverse_results, fill = TRUE)[metric == "cyclomatic_complexity",
        .(model, analysis = "Analysis 3 (reverse)", A = effect_A)],
    if (length(matched_results))
      rbindlist(matched_results, fill = TRUE)[metric == "cyclomatic_complexity",
        .(model, analysis = "Analysis 4 (matched)", A = effect_A)],
    fill = TRUE
  )
  p_A <- ggplot(effect_dt, aes(x = model, y = A, fill = analysis)) +
    geom_col(position = position_dodge(0.7), width = 0.6) +
    geom_hline(yintercept = A_THRESHOLD, linetype = "dashed") +
    coord_cartesian(ylim = c(0, 1)) +
    labs(title = "Effect size A on cyclomatic complexity",
         x = NULL, y = NULL, fill = NULL) +
    theme_minimal(base_size = 10) + theme(legend.position = "bottom")
  panels <- c(panels, list(p_A))
}

```

```
}  
cowplot::plot_grid(plotlist = panels, ncol = 1, align = "v")
```

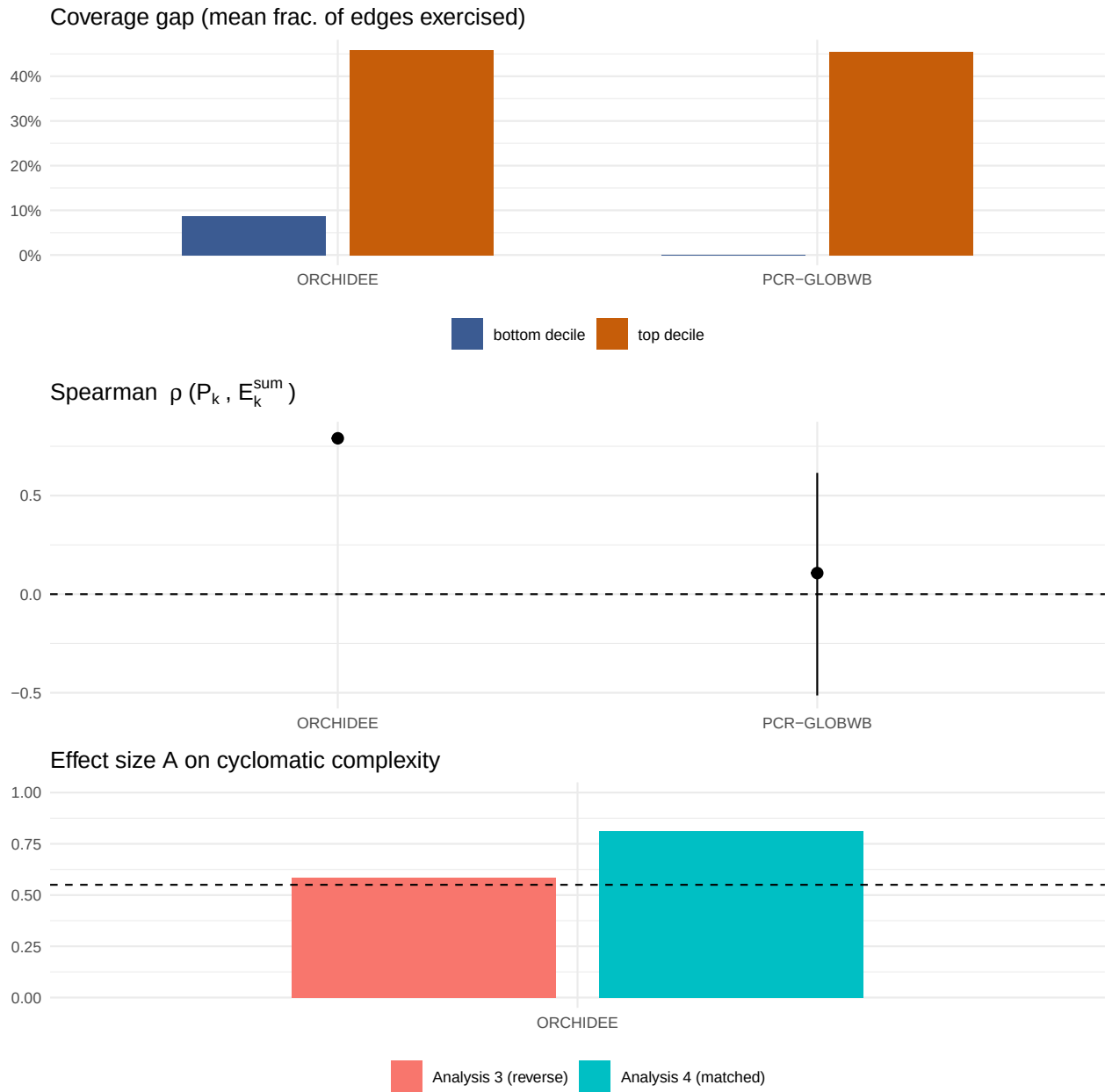


Figure 5: Cross-model synthesis: coverage gap (top vs bottom decile, mean fraction of edges exercised), Spearman $\rho(P_k, E_k^{\text{sum}})$ with bootstrap CI, and effect size A on cyclomatic complexity from Analyses 3 and 4. Dashed line at $A = 0.55$ is the pre-registered threshold.

11 Session information

```
# SESSION INFORMATION #####  
#####
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)  
## Platform: aarch64-apple-darwin20  
## Running under: macOS Tahoe 26.3.1  
##  
## Matrix products: default  
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/Libraries/libBLAS.dylib  
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  
##  
## locale:  
## [1] C  
##  
## time zone: Europe/London  
## tzcode source: internal  
##  
## attached base packages:  
## [1] tools parallel stats graphics grDevices utils datasets  
## [8] methods base  
##  
## other attached packages:  
## [1] effsize_0.8.1 MatchIt_4.7.2 boot_1.3-32  
## [4] softwareRisk_0.2.0 benchmarkme_1.0.8 sensobol_1.1.9  
## [7] ggraph_2.2.2 foreach_1.5.2 igraph_2.2.1  
## [10] tidygraph_1.3.1 here_1.0.2 tidytext_0.4.3  
## [13] ggrepel_0.9.6 readxl_1.4.5 cowplot_1.2.0.9000  
## [16] scales_1.4.0 openxlsx_4.2.8 lubridate_1.9.4  
## [19] forcats_1.0.1 stringr_1.6.0 dplyr_1.1.4  
## [22] purrr_1.2.1 readr_2.2.0 tidyr_1.3.2  
## [25] tibble_3.3.1 ggplot2_4.0.3.9000 tidyverse_2.0.0  
## [28] data.table_1.18.2.1  
##  
## loaded via a namespace (and not attached):  
## [1] tidymodels_1.2.1 viridisLite_0.4.3 farver_2.1.2  
## [4] viridis_0.6.5 S7_0.2.2 fastmap_1.2.0  
## [7] tweenr_2.0.3 janeaustenr_1.0.0 digest_0.6.39  
## [10] timechange_0.3.0 lifecycle_1.0.5 tokenizers_0.3.0  
## [13] magrittr_2.0.5 compiler_4.5.2 rlang_1.2.0  
## [16] yaml_2.3.12 knitr_1.51 labeling_0.4.3  
## [19] graphlayouts_1.2.2 RColorBrewer_1.1-3 withr_3.0.2  
## [22] grid_4.5.2 polyclip_1.10-7 iterators_1.0.14  
## [25] MASS_7.3-65 tinytex_0.58 cli_3.6.6  
## [28] rmarkdown_2.30 generics_0.1.4 otel_0.2.0
```



```
## [31] httr_1.4.8          tzdb_0.5.0          cachem_1.1.0
## [34] ggforce_0.5.0       splines_4.5.2       cellranger_1.1.0
## [37] vctrs_0.7.3         Matrix_1.7-4        hms_1.1.4
## [40] ineq_0.2-13         glue_1.8.1          benchmarkmeData_1.0.4
## [43] chk_0.10.0          codetools_0.2-20    stringi_1.8.7
## [46] gtable_0.3.6        pillar_1.11.1       htmltools_0.5.9
## [49] R6_2.6.1            Rdpack_2.6.6        doParallel_1.0.17
## [52] rprojroot_2.1.1     evaluate_1.0.5      lattice_0.22-7
## [55] backports_1.5.0     rbibutils_2.4.1     SnowballC_0.7.1
## [58] memoise_2.0.1       Rcpp_1.1.1-1.1      zip_2.3.3
## [61] nlme_3.1-168        gridExtra_2.3       mgcv_1.9-3
## [64] xfun_0.55           pkgconfig_2.0.3
```

```
cat("Machine:    "); print(get_cpu()$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
cat("Num cores:   "); print(detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```

```
cat("Num threads: "); print(detectCores(logical = TRUE))
```

```
## Num threads:
```

```
## [1] 10
```